```
    unique_lock(unique_lock const& ) = delete;
    unique_lock& operator=(unique_lock const& ) = delete;

    unique_lock(unique_lock&& );
    unique_lock& operator=(unique_lock&& );

    void swap(unique_lock& other) noexcept;

    void lock();
    bool try_lock();
    template<typename Rep, typename Period>
    bool try_lock_for(
        std::chrono::duration<Rep,Period> const& relative_time);
    template<typename Clock, typename Duration>
    bool try_lock_until(
        std::chrono::time_point<Clock,Duration> const&
absolute_time);
    void unlock();

    explicit operator bool() const noexcept;
    bool owns_lock() const noexcept;
    Mutex* mutex() const noexcept;
    Mutex* release() noexcept;
};
```

Std::Unique_Lock Default Constructor

Constructs a `std::unique_lock` instance with no associated mutex.

Declaration

```
unique_lock() noexcept;
```

Effects

Constructs a `std::unique_lock` instance that has no associated mutex.

Postconditions

```
this->mutex()==NULL, this->owns_lock()==false.
```

Std::Unique_Lock Locking Constructor

Constructs a `std::unique_lock` instance that locks the supplied mutex.